

Array

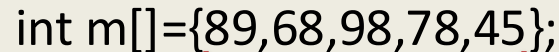
- **Array** is the collection of similar type of data with contiguous memory allocation.

Without Array

```
int m1=89;  
int m2=68;  
int m3=98;  
int m4=78;  
int m5=45;
```

With Array

```
int m[]={89,68,98,78,45};
```



The diagram shows the declaration `int m[]={89,68,98,78,45};`. A red arrow points from the variable `m` to the label "Array Variable". Another red arrow points from the curly braces `{ }` to the label "Array Elements".

Array Variable

Array Elements

Why Array is a static Data Structure?

Array size is fixed. Size of array can not be increase or decrease, without creating new array.

Steps for Array Creation

Step 1- Array Declaration

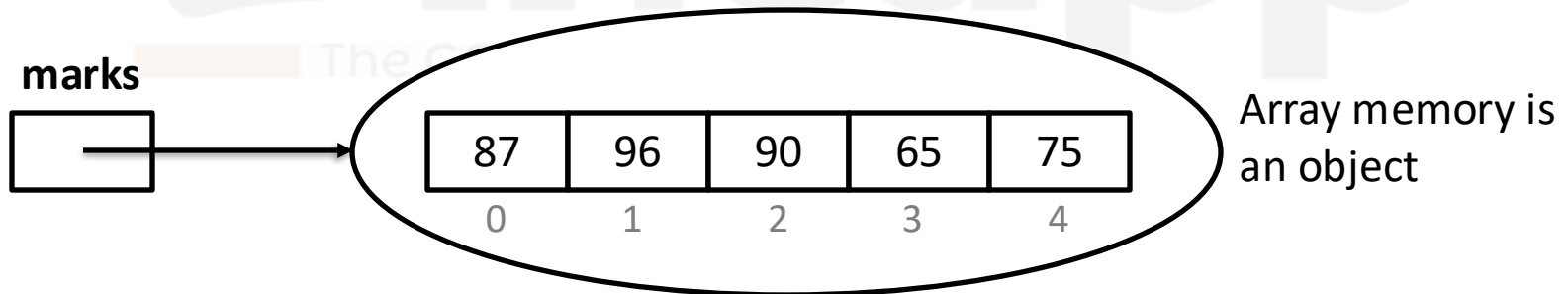
```
int marks[]; or int []marks;
```

Step 2- Array Memory Creation

```
marks=new int[5];
```

Step 3- Array Initialization:

```
marks[0]=87;  
marks[1]=96;  
marks[2]=90;  
marks[3]=65;  
marks[4]=75;
```



Note:

Array indexing starts from 0. Because by default array variable refers the first block of the memory. That's why to access first block, we need to add 0 to current address and to access second block, we need to add 1.

Array Creations

Array Creation:

```
int marks[]= new int[5];  
marks[0]=87;  
marks[1]=96;  
marks[2]=90;  
marks[3]=65;  
marks[4]=75;
```



Array Creation:

```
int marks[]= new int[ ]{87,96,90,65,75};
```



Array Creation:

```
int marks[]= new int[5]{87,96,90,65,75};
```



Array Creation:

```
int marks[]={87,96,90,65,75};
```



Array Creation:

```
int marks[];  
marks={87,96,90,65,75};
```



Array Creation:

```
int marks[];  
marks=new int[ ]{87,96,90,65,75};
```



Accessing an Array



We can access array using two types of loop controls:

1. Normal for loop
2. for-each loop

Normal for loop

```
int marks[]={89,68,98,78,45};  
for(int i=0; i<marks.length ; i++) {  
    System.out.println(marks[i]);  
}
```

For-Each loop

```
int marks[]={89,68,98,78,45};  
for(int x : marks) {  
    System.out.println(x);  
}
```

Note:

‘marks.length’ property return the number of elements in array.

Operations on Array

The operations that can be performed on array are:

Traversing	Visiting every element
Insertion	Adding a new element
Deletion	Removing the existing element
Searching	Checking the existence of a particular element
Sorting	Arranging the elements in a particular order
Merging	Combining two or more arrays

Note: I will explain the insertion, traversing and deletion operation.
All the remaining operations will be explained in chapters of Searching and Sorting.

Traversing Operation on Array

Algo for Traversing an Array

Let **arr** is array:

1. Create array **arr**.
2. Read array **arr**.
3. Check, if (**arr.length** > 0), then:
 - a) Repeat for $i = 0$ to (**arr.length** - 1)
Print, **arr[i]**
4. Otherwise,
 - a) Print, No element to traverse.
5. Exit.

Insertion Operation on Array

Algo for Insertion at End of Array

Let **arr** and **newArr** are the arrays, **value** will store the value to be inserted, then the algorithm is given below:

1. Create and Read array **arr**.
2. Read **value**.
3. Create new array **newArr** with one more element.
4. Copy all elements of old array to new array.

Repeat for $i = 0$ to $(\text{arr.length} - 1)$

- Set **newArr[i]=arr[i]**

5. Set **newArr[newArr.length-1] = value**.
6. Exit.

Insertion Operation on Array

Algo for Insertion at Beginning of Array

Let **arr** and **newArr** are the arrays, **value** will store the value to be inserted, then the algorithm is given below:

1. Create and Read array **arr**.
2. Read **value**.
3. Create new array **newArr** with one more element.
4. Set **newArr[0] = value**.
5. Copy all elements of old array to new array.
Repeat for $i = 0$ to $(arr.length - 1)$
 - Set **newArr[i+1]=arr[i]**
6. Exit.

Insertion Operation on Array

Algo for Insertion at Specified index of Array

Let **value** will store the value to be inserted and **index** will be that specified index of array where the value to be inserted:

1. Create and Read array **arr**
2. Read **value** & **index**.
3. Check if **index** ≥ 0 and **index** $\leq (\text{arr.length}-1)$
 - a) Create new array **newArr** with one more element.
 - b) Set **newArr[index] = value**
 - c) Repeat for $i = 0$ to $(\text{newArr.length} - 1)$
 - i. Check, if $(i < \text{index})$, then
 - Set **newArr[i] = arr[i]**
 - ii. Otherwise,
 - Set **newArr[i+1] = arr[i]**
4. Otherwise,
 - a) Print, invalid index.
5. Exit.

Deletion Operation on Array

Algo for Deletion from End of Array

Let **arr** and **newArr** are the arrays, then the algorithm is given below:

1. Create array **arr**.
2. Read array **arr**.
3. Create new array **newArr** with one less element.
4. Copy all elements of old array (except last one) to new array
Repeat for $i = 0$ to $(arr.length - 2)$
 - Set **newArr[i]=arr[i]**
5. Exit.

Deletion Operation on Array

Algo for Deletion from Beginning of Array

Let **arr** and **newArr** are the arrays, then the algorithm is given below:

1. Create array **arr**.
2. Read array **arr**.
3. Create new array **newArr** with one less element.
4. Copy all elements of old array (except first one) to new array
Repeat for $i = 1$ to $(arr.length - 1)$
 - Set **newArr[i-1]=arr[i]**
5. Exit.

Deletion Operation on Array

Algo for Deletion from Specified index of Array

Let **arr** and **newArr** are the arrays, and **index** will be that specified index of array where the vale to be deleted, **i** is the index for iteration:

1. Create and Read array **arr**.
2. Read **index**.
3. Check if $\text{index} \geq 0$ and $\text{index} \leq (\text{arr.length} - 1)$
 - a) Create new array **newArr** with one less element.
 - b) Repeat for $i = 0$ to $(\text{newArr.length} - 1)$
 - i. Check, if $(i < \text{index})$, then
 - Set **newArr[i]=arr[i]**
 - ii. Otherwise,
 - Set **newArr[i]=arr[i+1]**
4. Otherwise,
 - a) Print, invalid index.
5. Exit.